

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /***** /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ *****/ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> /***** /* Listing Contents: /* Reuse instructions /* Modification instructions /* Compilation instructions /* Includes /* Class declarations: /* CData /* ASet /* Source code in c:/classes/CData.cpp: /* CData /* CData() /* Empty() *****/ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> /***** /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ *****/ </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	<pre> Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */ </pre>
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	<pre>If(record_count > limit) /* have all records been processed? */</pre>
Bad Comment	<pre>If(record_count > limit) /* check if record count exceeds limit */</pre>
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> /***** /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ *****/ </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> while (miss_distance > threshold) { success_code = move_robot (target_location); if (success_code == MOVE_FAILED) { printf("The robot move has failed.\n"); } } </pre>
Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	<pre> #define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS; </pre>

Evaluation criteria and suggestions

Evaluation criteria

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar el desarrollo de programas Java consistentes con los estándares PSP.
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /***** * Programa: Program 6 - Encontrar x para Distribución t-Student * Clase: App * Autor: [Karla Sofía Castro Pérez] * Fecha: [10-12-2025] * * Descripción: * Punto de entrada principal del programa Program 6. Crea una instancia * de la clase Logic y ejecuta el procesamiento principal para encontrar * el valor x tal que la integral de 0 a x de la distribución t-Student * sea igual a un valor p dado. * * Compilación: * javac App.java Logic.java SearchAlgorithm.java SimpsonIntegration.java * TDistribution.java GammaFunction.java Data.java * * Ejecución: * java App * * Reuse Instructions: * - Esta clase solo debe ser modificada para cambios en el flujo principal * - El manejo de excepciones ya está implementado * - Para cambios en la lógica, modificar la clase Logic * * Notas: * - Requiere Java 8 o superior * - Todos los archivos .java deben estar en mismo directorio *****/ </pre>
Listing Contents	Cada archivo debe estar estructurado claramente.
Contents Example	Estructura del archivo (ejemplo de Logic5a.java): ○ Comentario de licencia/encabezado ○ Imports organizados ○ Comentario de clase con atributos ○ Métodos públicos organizados ○ Métodos privados organizados
Reuse Instructions	• Documentar claramente cómo usar cada clase. Contenido requerido: ○ Propósito de la clase ○ Cuándo modificar ○ Dependencias con otras clases ○ Limitaciones conocidas

Reuse Example	* Reuse Instructions: ○ * Esta clase coordina el flujo del programa ○ * Modificar solo si cambia el flujo principal ○ * Para cambios en el algoritmo de búsqueda, modificar SearchAlgorith
Identifiers	• Variables: Usar nombres descriptivos con prefijos tipo (ej: dblIPTarget, intlteraciones) • Clases: PascalCase (ej: SearchAlgorithm, SimpsonIntegration) • Métodos: camelCase (ej: executeInteractiveMode(), calculateCoefficient()) • Constantes: UPPER_CASE con guiones bajos (ej: MAX_ITERATIONS)
Identifier Example	Buenos ejemplos (del código): • <code>private double dblAcceptableError; //</code> • <code>private SimpsonIntegration objIntegrator; //</code> • <code>public double calculateIntegral(int intNumSeg) //</code> Malos ejemplos (NO permitidos): • <code>double x; //</code> ✗POCO DESCRIPTIVO • <code>SimpsonIntegration s; //</code> ✗ABREVIACIÓN • <code>public double calc(int n) //</code> ✗ABREVIADO

(continued)

Coding Standard Template (continued)

Comments	• Comentarios de bloque: Para secciones principales • Comentarios de línea: Para explicar lógica compleja • Comentarios Javadoc: Para todos los métodos públicos
Good Comment	<pre>/** * Valida todos los parámetros de entrada del constructor. * * @param dbIPTarget valor p objetivo * @param intDof grados de libertad * @param dbIAcceptableError error aceptable * * @throws IllegalArgumentException si algún parámetro es inválido */</pre>
Bad Comment	<pre>/** * Parámetros del constructor. * */</pre>
Major Sections	Separar visualmente secciones principales del código.
Example	<pre>/*-----*/ /* Attributes */ /*-----*/ private double dblX; /*-----*/ /* Constructor */ /*-----*/ public ClassName() { ... } /*-----*/ /* Public Methods */ /*-----*/ public void method() { ... } /*-----*/ /* Private Methods */ /*-----*/ private void helperMethod() { ... }</pre>
Blank Spaces	• Una línea en blanco entre métodos • Dos líneas en blanco entre secciones principales • Espacios alrededor de operadores: <code>dblX = dblY + dblZ</code> • Espacios después de comas en parámetros: <code>(dblX, intDof, dblError)</code>

Indenting	• 4 espacios por nivel de indentación (NO tabs) • Llaves en líneas separadas alineadas verticalmente • Contenido indentado dentro de llaves
Indenting Example	<pre>try { // 4 espacios de indentación objLogic = new Logic5a(); // Instanciar objeto Logic // Método indentado objLogic.execute(); // Llamar al método de ejecución principal } catch (IllegalArgumentException objEx) { // Catch alineado con try System.err.println("\n" + "=" .repeat(60)); }</pre>
Capitalization	• Clases: PascalCase (SearchAlgorithm) • Métodos y variables: camelCase (findX, dblResultado) • Constantes: UPPER_CASE (MAX_ITERATIONS) • Paquetes: lowercase (no se usan en este proyecto)
Capitalization Example	<pre>public class SimpsonIntegration // PascalCase para clases { private double dblAcceptableError; // camelCase con prefijo tipo public double calculateIntegral() // camelCase para métodos private static final int MAX_ITERATIONS = 100; // UPPER_CASE para constantes }</pre>